

Trees and forests

Machine Learning in Finance · Session 11

The models so far

	model	defined by	test score
a number	linear regression (OLS)	$\min_{\beta} \sum_i (y_i - \beta_0 - \sum_j \beta_j x_{ij})^2$	RMSE 0.237 on one column, 0.248 on 19
	ridge	$\min_{\beta} \text{RSS} + \alpha \sum_j \beta_j^2$	RMSE 0.218
	lasso	$\min_{\beta} \text{RSS} + \alpha \sum_j \beta_j $	RMSE 0.217
a label	logistic regression	$P(y = 1) = \sigma(\beta_0 + \sum_j \beta_j x_j)$	AUC 0.748
	k-nearest neighbours	$\hat{p} = \frac{1}{k} \sum_{k \text{ nearest}} y_i$	AUC 0.749, $k = 25$

Every model so far fits one formula to the whole table, or averages the nearest rows. Today's model asks yes-or-no questions instead, with the cuts chosen from the data.

The volatility table

Python Code

```
from sklearn.metrics import mean_squared_error

table = pd.read_csv("data/market_features.csv", parse_dates=["date"]).set_index("date")
train, test = table.loc[:"2022-12-31"], table.loc["2023-01-01":]
columns = list(table.columns[:-1])
print(len(train), len(test), len(columns))

def rmse(actual, predicted):
    return np.sqrt(mean_squared_error(actual, predicted))
```

1894 482 19

One row per trading day, and **vol_next** is the index's volatility over the next 20 days. The split, the 19 columns and **rmse** are the ones ridge and lasso were scored with.

Today

- where a **linear model** stops on the volatility table
- a **tree**: questions, boxes and a formula
- **growing** a tree, one question at a time
- trees in scikit-learn, and **drawing** them with `plot_tree`
- how **deep** to grow a tree, and pruning
- **bagging** and **random forests**: many trees, averaged
- **tuning** a forest, what a search **costs**, and model **size**
- a tree for a **label**, on the credit table

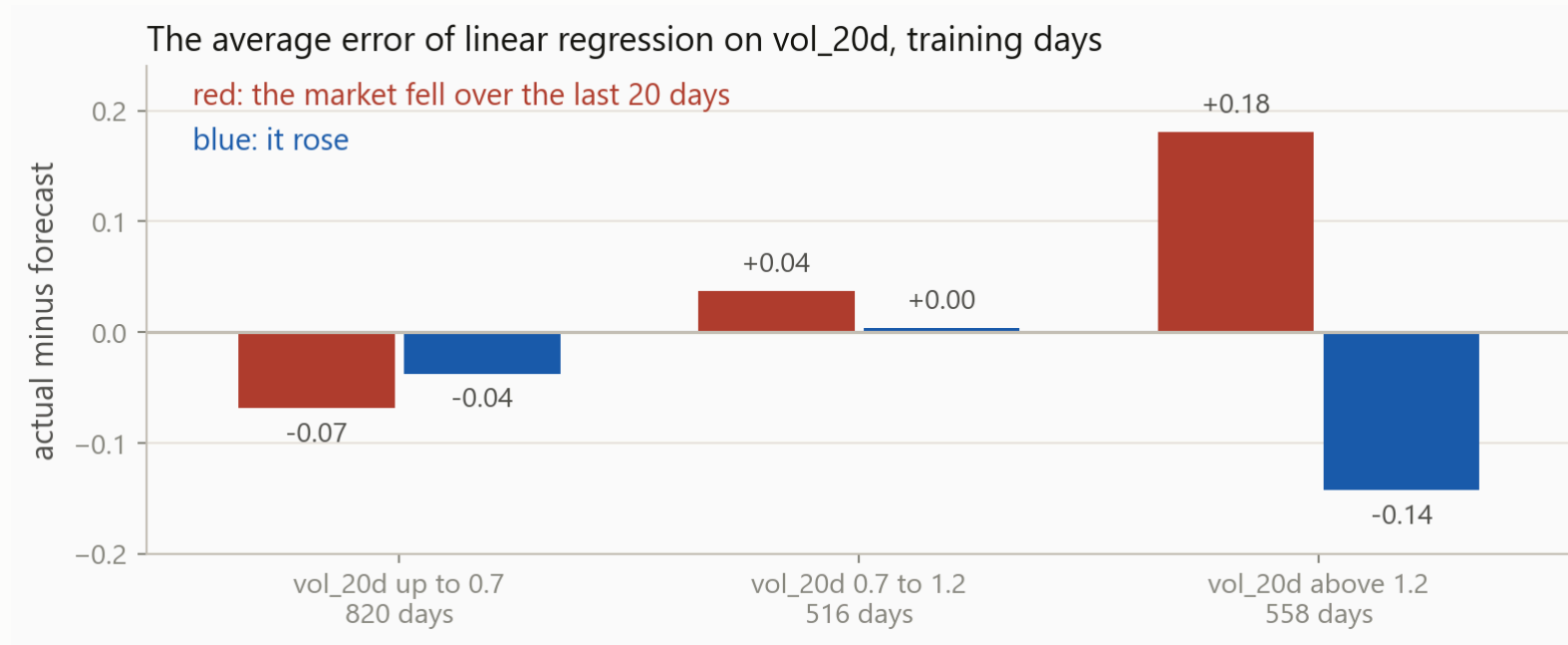
The exercises and the case are where you practise each of these.

1 • Where a linear model stops

PART 1

A pattern in the volatility table that one coefficient per column cannot follow.

The error of linear regression, by level and direction



When **vol_20d** is above 1.2, linear regression is 0.18 too low after a 20-day fall and 0.14 too high after a rise. It gives both kinds of day the same forecast.

What a line can fit

What the table shows

When volatility is high, the next 20 days stay high after a fall and calm down after a rise. How much `vol_20d` matters depends on the market's direction.

What a linear model can fit

One coefficient per column, the same after a fall and after a rise. A different slope for falls has to be written in by hand as a new column, at a cut already known.

A model of this table has to ask about the level first, and then about the direction when the level is high, with the cuts placed by the data.

2 · A tree of questions

PART 2

What a tree is, how it forecasts, and how to write it down.

A decision tree

the root

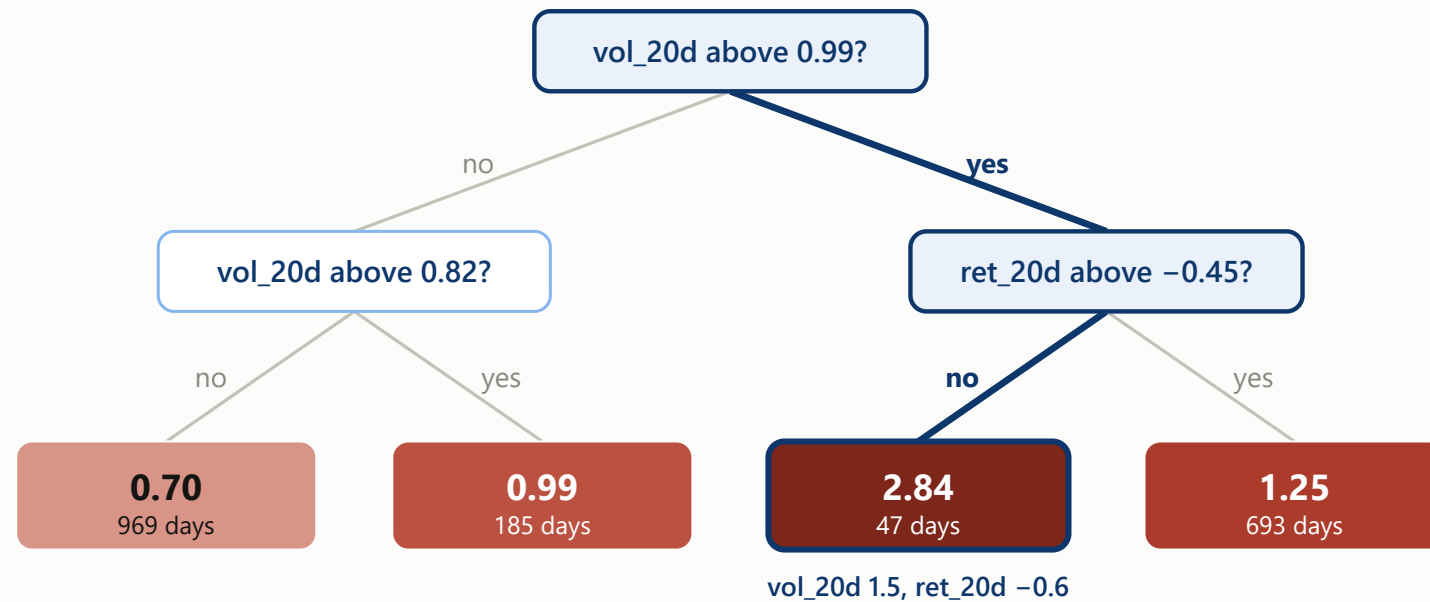
depth 0: the first question

a question

depth 1: asked inside one box

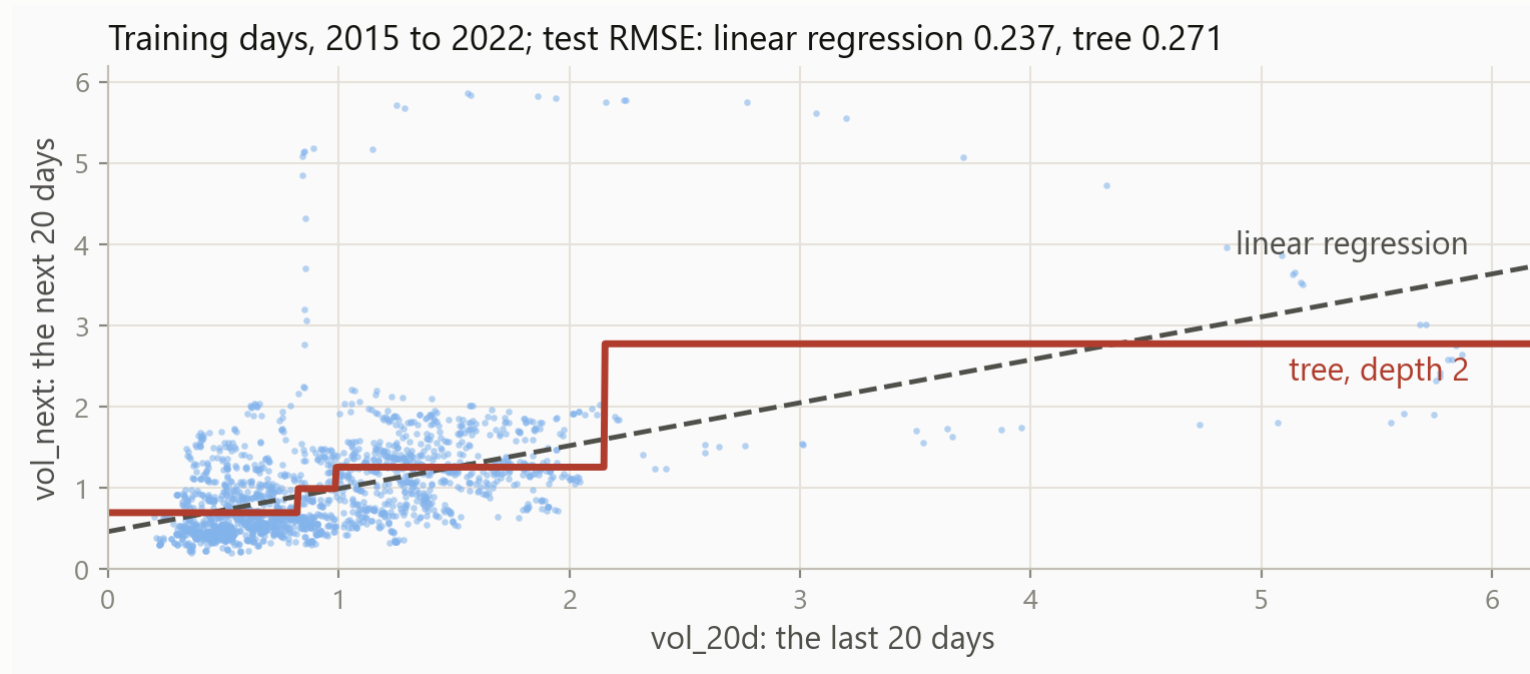
the leaves

depth 2: the boxes at the end



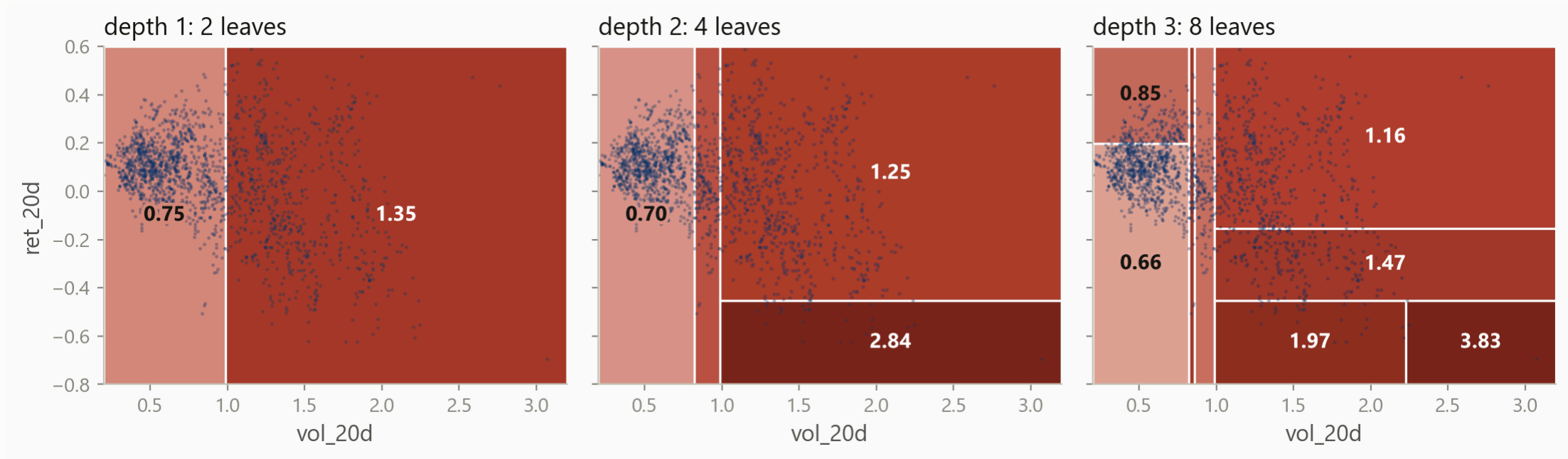
A day answers the questions from the root down and lands in one leaf. A day with **vol_20d** at 1.5 and **ret_20d** at -0.6 gets 2.84: the average of 47 training days.

A tree on one column



On `vol_20d` alone, a tree of depth 2 cuts at 0.82, 0.99 and 2.15 and forecasts four flat steps. On the test days it scores 0.271, linear regression 0.237.

The same tree as boxes



With two columns each question cuts the plane in two, and each level cuts every box again. Each box is shaded by its forecast; the dots are the training days.

A tree as a formula

$$\hat{f}(\mathbf{x}) = \sum_{m=1}^M c_m \mathbf{1}\{\mathbf{x} \in R_m\}$$

The leaves are boxes $R_1, \dots, R_M$, and c_m is the mean **vol_next** of the training days in box m . The indicator picks out the one box holding $\mathbf{x}$.

In the tree of depth 2, $M = 4$, and the four c_m are 0.70, 0.99, 2.84 and 1.25.

Three ways to forecast

model	a forecast is	it needs
linear regression	one straight formula for the whole table	the shape written in: a column for every bend or interaction
k-nearest neighbours	the average of the k closest training rows	a scaler, and a distance that suits the columns
a tree	the average of the training rows in the same box	enough rows to place the cuts

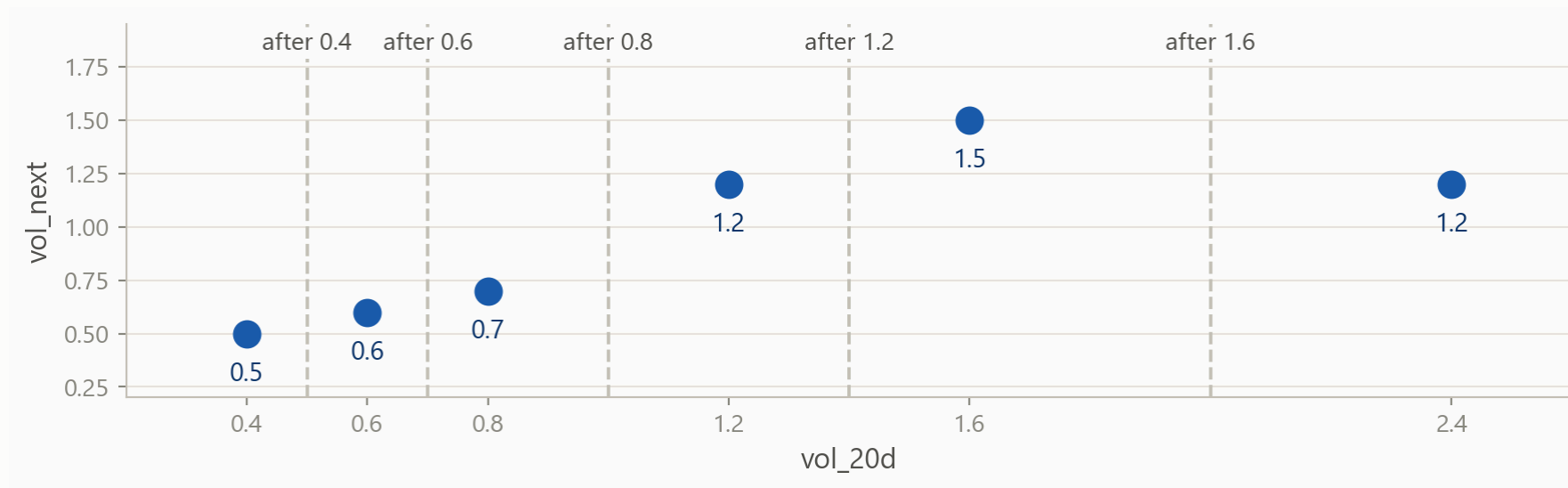
A tree forecasts from nearby training rows, as k-nearest neighbours does, but its neighbourhoods are boxes placed by the data, and a scaler changes nothing.

3 • Growing a tree

PART 3

How the questions are chosen, one at a time.

Six days



The six are real training days, rounded to one decimal. A question on `vol_20d` can cut them in five places; the best cut leaves each box with days as alike as possible.

The best number for one box

A box forecasts one number, c , for all its days. The squared error is smallest where its slope in c is zero:

$$\frac{d}{dc} \sum_i (y_i - c)^2 = -2 \sum_i (y_i - c) = 0 \implies c = \bar{y}$$

The best forecast is the box's mean, and what is left is its RSS, $\sum_i (y_i - \bar{y})^2$. Three days at 0.5, 0.6 and 0.7 forecast 0.6 and leave 0.02.

Every cut on the six

cut after	left box	right box	total RSS
no cut		6 days, mean 0.95: 0.815	0.815
0.4	1 day, mean 0.5: 0	5 days, mean 1.04: 0.572	0.572
0.6	2 days, mean 0.55: 0.005	4 days, mean 1.15: 0.330	0.335
0.8	3 days, mean 0.6: 0.020	3 days, mean 1.3: 0.060	0.080
1.2	4 days, mean 0.75: 0.290	2 days, mean 1.35: 0.045	0.335
1.6	5 days, mean 0.9: 0.740	1 day, mean 1.2: 0	0.740

Each row adds up the RSS of the two boxes. The cut after 0.8 leaves 0.080, far below the rest: it separates the calm days from the busy ones.

The rule, in general

$$\min_{j, s} \left[\sum_{i: x_{ij} \leq s} (y_i - \bar{y}_L)^2 + \sum_{i: x_{ij} > s} (y_i - \bar{y}_R)^2 \right]$$

Column j and cut s make the two boxes, with means $\bar{y}_L$ and $\bar{y}_R$. It is the RSS of least squares, with one constant per box in place of a line.

scikit-learn tries every column and every cut, and keeps the smallest.

The six, in code

Python Code

```
six = pd.DataFrame({"vol_20d": [0.4, 0.6, 0.8, 1.2, 1.6, 2.4],
                    "vol_next": [0.5, 0.6, 0.7, 1.2, 1.5, 1.2]})

def rss(y):
    return ((y - y.mean()) ** 2).sum()

left = six[six["vol_20d"] <= 0.8]
right = six[six["vol_20d"] > 0.8]
print(round(rss(left["vol_next"]), 3), round(rss(right["vol_next"]), 3))
```

0.02 0.06

rss adds up each value's squared distance from the mean of its box. The cut after 0.8 leaves 0.02 on the left and 0.06 on the right, as in the table.

Every cut on vol_20d

Python Code

```
print(round(rss(train["vol_next"]), 1))  
for cut in [0.5, 0.75, 1.25, 1.5, 2.0, 3.0]:  
    left = train[train["vol_20d"] <= cut]["vol_next"]  
    right = train[train["vol_20d"] > cut]["vol_next"]  
    print(cut, round(rss(left) + rss(right), 1))
```

908.5

0.5 841.7

0.75 763.1

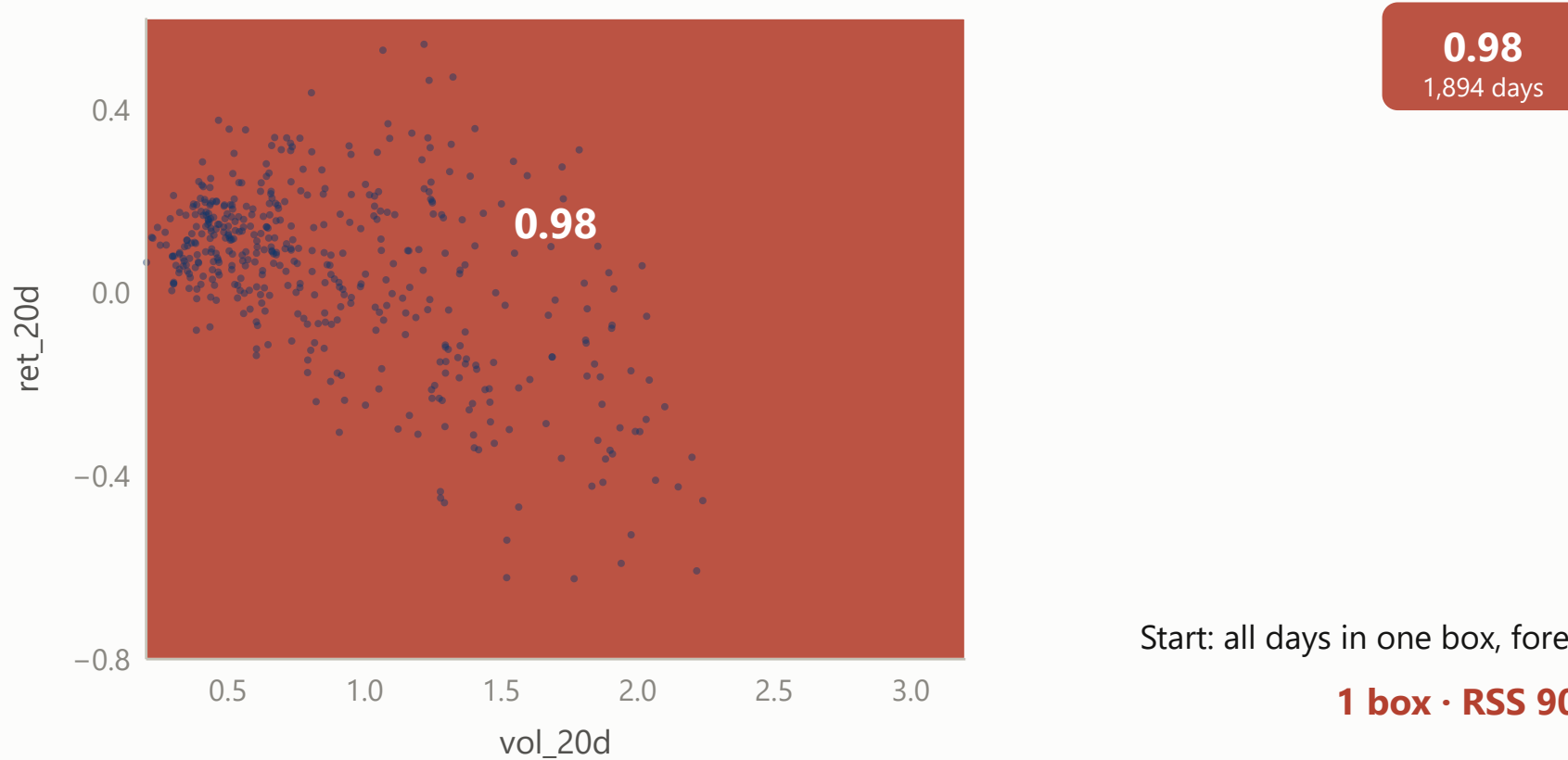
1.25 752.1

1.5 766.4

2.0 769.9

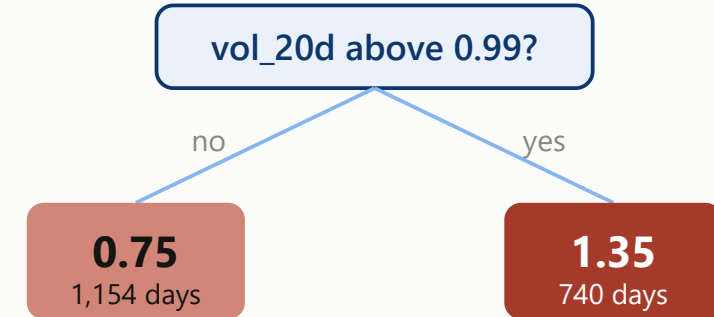
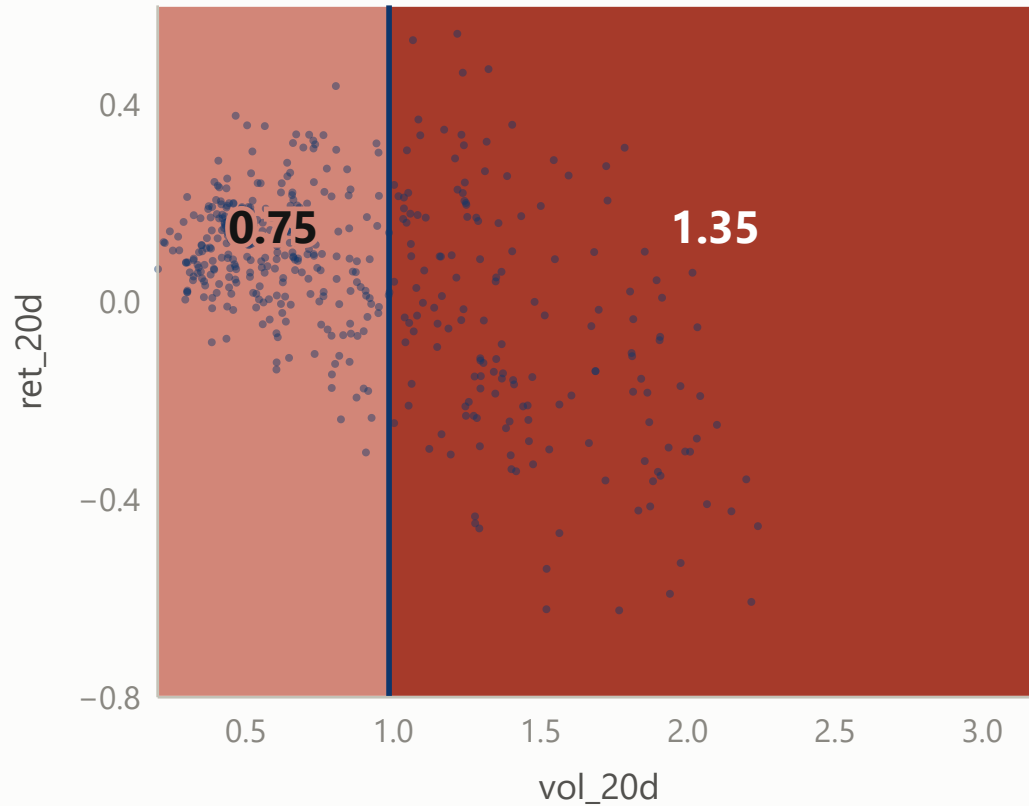
3.0 802.3

Recursive binary splitting, step by step



Each step keeps the cut with the smallest RSS inside one box, and earlier cuts are never revised: a greedy search. Three questions take the RSS from 908.5 to 617.6.

Recursive binary splitting, step by step

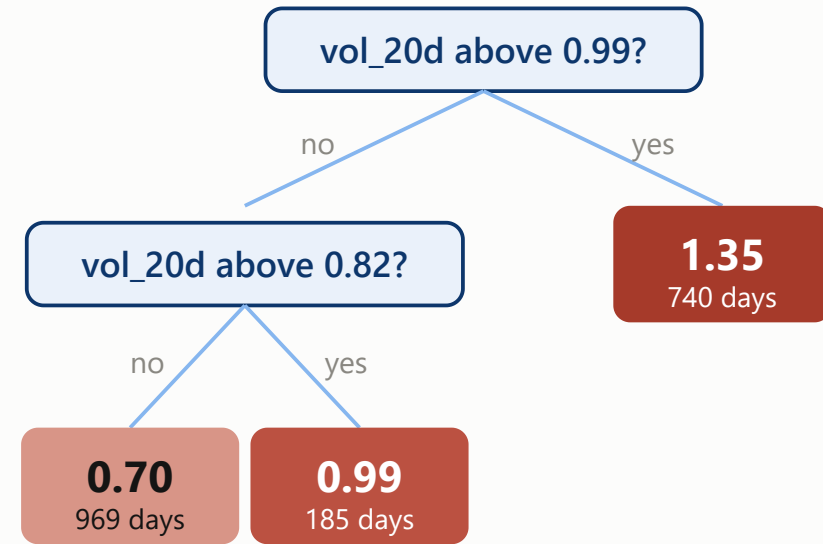
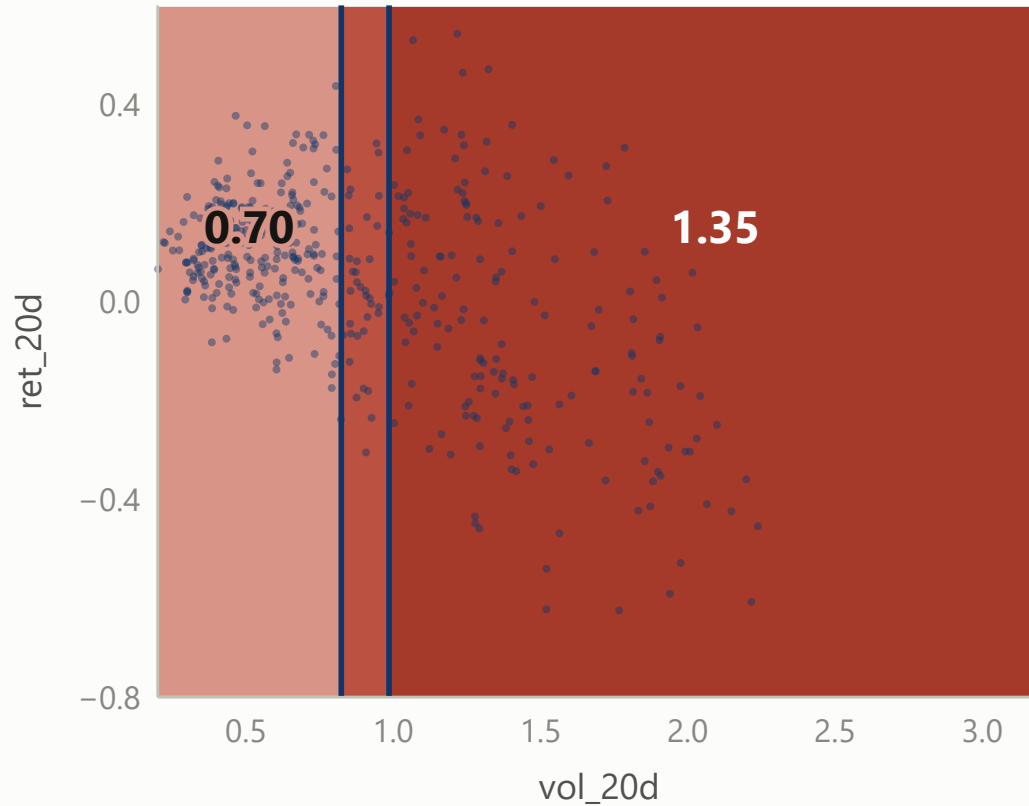


Try every column and cut; keep the smallest RSS.

2 boxes · RSS 742.2

Each step keeps the cut with the smallest RSS inside one box, and earlier cuts are never revised: a greedy search. Three questions take the RSS from 908.5 to 617.6.

Recursive binary splitting, step by step

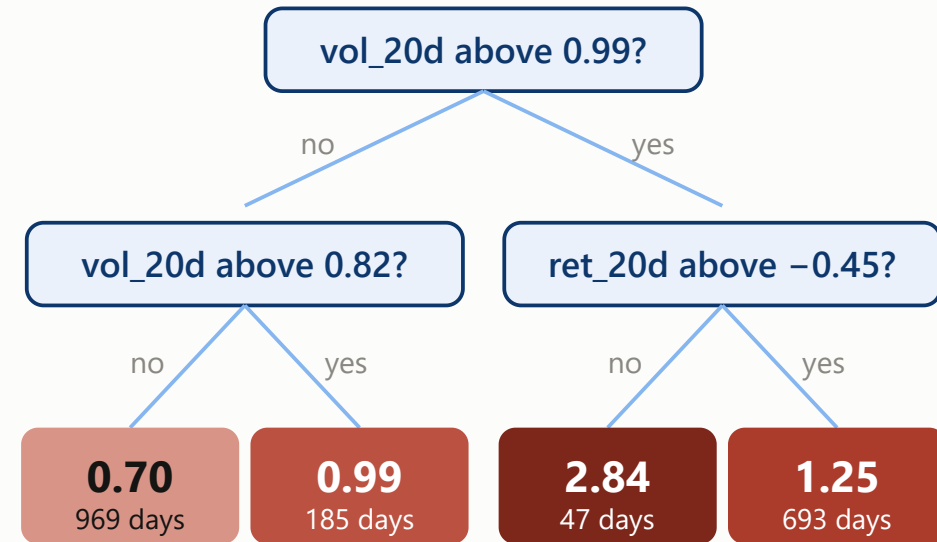
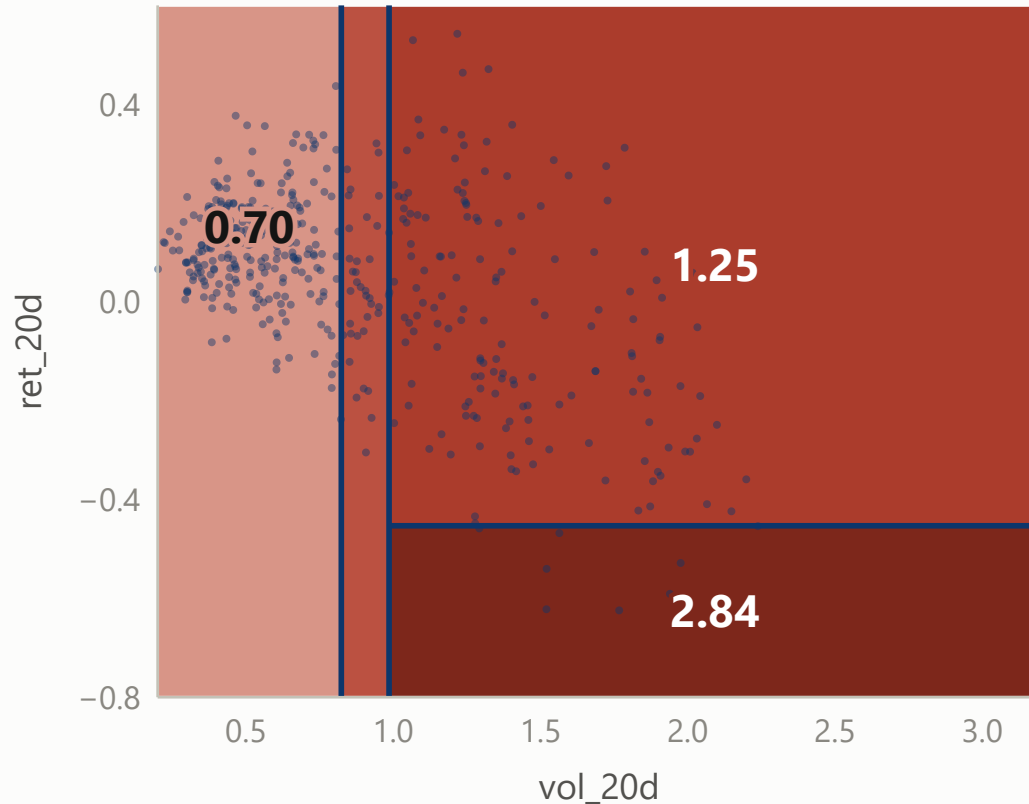


Search again inside the calm box.

3 boxes · RSS 728.6

Each step keeps the cut with the smallest RSS inside one box, and earlier cuts are never revised: a greedy search. Three questions take the RSS from 908.5 to 617.6.

Recursive binary splitting, step by step



And inside the busy box: the direction matters.

4 boxes · RSS 617.6

Each step keeps the cut with the smallest RSS inside one box, and earlier cuts are never revised: a greedy search. Three questions take the RSS from 908.5 to 617.6.

4 • Trees in scikit-learn

PART 4

Fitting a tree, drawing it with `plot_tree`, and reading its forecasts.

The best cut, found by scikit-learn

Python Code

```
from sklearn.tree import DecisionTreeRegressor, plot_tree
pair = ["vol_20d", "ret_20d"]
stump = DecisionTreeRegressor(max_depth=1, random_state=0)
stump.fit(train[pair], train["vol_next"])
plt.figure(figsize=(10, 3))
plot_tree(stump, feature_names=pair, fontsize=10)
plt.show()
```

Reading one box

line	in the top box	what it says
the question	<code>vol_20d <= 0.986</code>	True goes left, False right; <code>vol_20d</code> beat <code>ret_20d</code>
<code>squared_error</code>	0.48	the box's RSS divided by its days: the variance of <code>vol_next</code>
<code>samples</code>	1894	the training days in the box
<code>value</code>	0.983	the mean of <code>vol_next</code> in the box: its forecast

The two leaves forecast 0.745 for the 1,154 calmer days and 1.353 for the 740 busier ones. Each leaf's `value` is its c_m .

Your turn 1

Keep only the busy days, with `vol_20d` above 0.986, and fit a stump to them. Which question does it ask?

Python Code

```
busy = ...
busy_stump = DecisionTreeRegressor(max_depth=1, random_state=0)
...
plt.figure(figsize=(10, 3))
plot_tree(busy_stump, feature_names=pair, fontsize=10)
plt.show()
```

A mask keeps the rows where a condition holds, as in the loop over cuts. Fit on `busy[pair]` and `busy["vol_next"]`.

Your turn 1: answer

```
busy = train[train["vol_20d"] > 0.986]  
busy_stump.fit(busy[pair], busy["vol_next"])
```

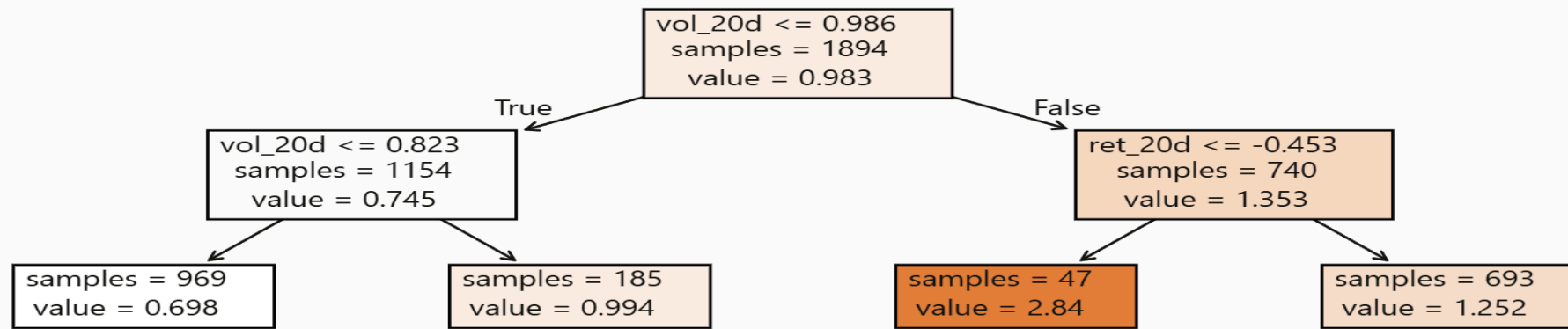
Inside the busy box the best question is about `ret_20d`. After 20 days that fell 0.45 percent a day or more, the next 20 averaged 2.84; otherwise 1.25.

This is how a tree finds an interaction: each question is chosen for the rows of one box. A tree of depth 2 does it for every box.

A tree of depth 2

Python Code

```
small = DecisionTreeRegressor(max_depth=2, random_state=0)
small.fit(train[pair], train["vol_next"])
plt.figure(figsize=(10, 3))
plot_tree(small, feature_names=pair, filled=True, impurity=False, fontsize=10)
plt.show()
```



Forecasts from the leaves

Python Code

```
p_small = small.predict(test[pair])  
  
print(np.unique(p_small).round(3))  
print(round(rmse(test["vol_next"], p_small), 4))
```

```
[0.698 0.994 1.252]  
0.2691
```

The 482 test days get only three different forecasts: none had high volatility after a steep fall. The error is 0.269, against 0.237 for linear regression.

A tree on all 19 columns

Python Code

```
tree = DecisionTreeRegressor(max_depth=3, random_state=0)
tree.fit(train[columns], train["vol_next"])

print(round(rmse(test["vol_next"], tree.predict(test[columns])), 4))
```

0.3903

There is no scaler, because a cut compares one column with one number. Three levels of questions on all 19 columns score 0.390, worse than guessing the training average, 0.291.

Your turn 2

Draw **tree** in the clearer style. Which columns does it ask about, and why might that go wrong on the test days?

Python Code

```
plt.figure(figsize=(12, 3.4))
plot_tree(tree, ...)
plt.show()
```

The column names are in **columns**. **label="root"** names the fields in the top box only, and **precision=2** and **fontsize=9** keep the text small.

Your turn 2: answer

```
plt.figure(figsize=(12, 3.4))  
plot_tree(tree, feature_names=columns, filled=True, impurity=False,  
          label="root", precision=2, fontsize=9)  
plt.show()
```

The second question is Disney's volatility above 1.26. It was on 330 of the 482 test days, so they got 1.20, while the index's next 20 days averaged 0.80.

The first question sets aside the 27 most volatile training days, 26 of them in 2020: squared errors weigh the biggest misses most.

5 • How deep to grow

PART 5

A tree without a limit ends with a leaf for every training day.

The training days, scored **PREDICT**

Grow a tree with no limit on `vol_20d` and `ret_20d`, and score it on the days it was grown on; the depth-2 tree scores 0.571 there. What will the error be?

Python Code

```
full = DecisionTreeRegressor(random_state=0)
full.fit(train[pair], train["vol_next"])

print(round(rmse(train["vol_next"], full.predict(train[pair])), 4))
```

A tree without a limit

Python Code

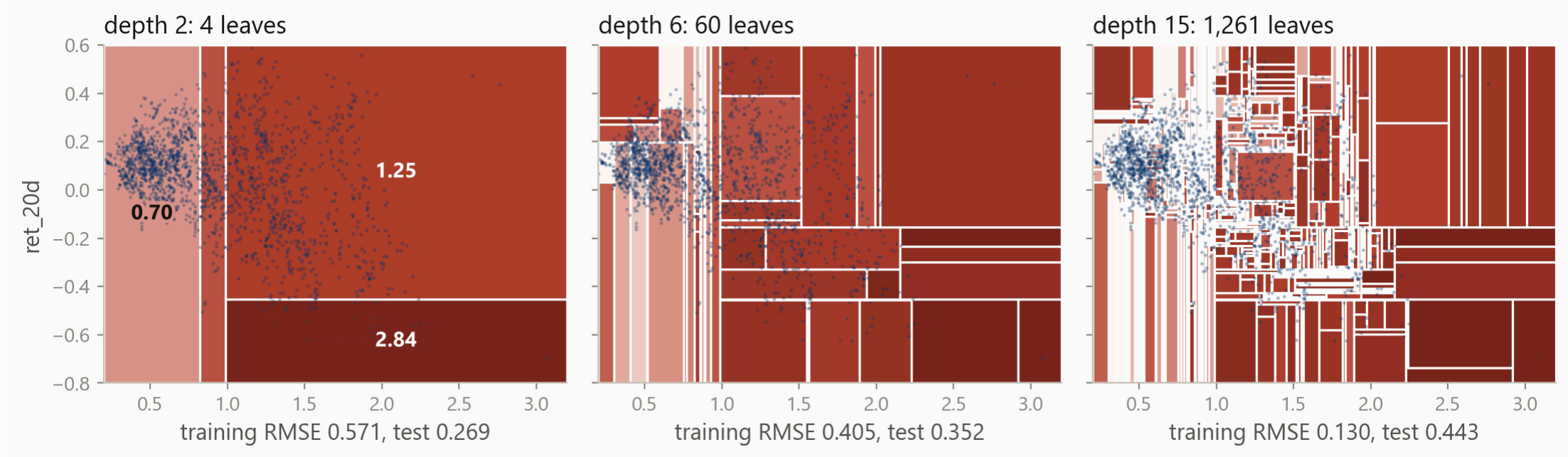
```
full = DecisionTreeRegressor(random_state=0)
full.fit(train[pair], train["vol_next"])

print(full.get_depth(), full.get_n_leaves())
print(round(rmse(test["vol_next"], full.predict(test[pair])), 4))
```

```
32 1894
0.4663
```

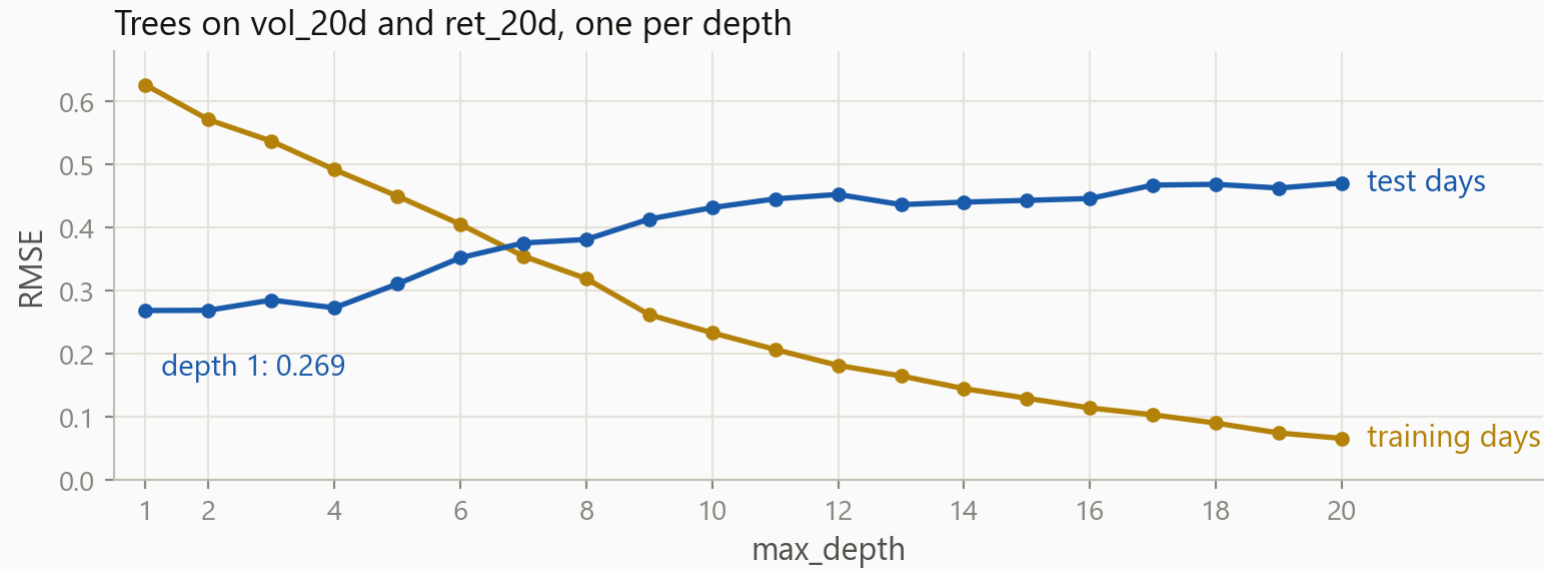
The tree asks up to 32 questions and ends with 1,894 leaves, one for every training day. On the test days its error is 0.466, against 0.269 at depth 2.

Boxes around single days



The same plane is cut at three depths. At depth 15 there are 1,261 boxes for 1,894 days: the training error is 0.130, the test error 0.443.

Depth against RMSE



The test error is lowest at depth 1 or 2, 0.269, and climbs to 0.471 by depth 20. The training error falls towards 0.

Depth moves a tree along the bias-variance trade-off: a shallow tree is biased, and a deep one changes with every sample of days.

Depth on the folds

Python Code

```
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit

folds = TimeSeriesSplit(n_splits=5)
grid = {"max_depth": [1, 2, 3, 4, 6, 8]}
search = GridSearchCV(DecisionTreeRegressor(random_state=0), grid, cv=folds,
                      scoring="neg_root_mean_squared_error")
search.fit(train[pair], train["vol_next"])
print(search.best_params_, round(-search.best_score_, 4))
```

```
{'max_depth': 2} 0.6368
```

TimeSeriesSplit scores each fold on later days, as it did for ridge's alpha; the folds include 2020, so their errors are larger. Depth 2 wins, at 0.637.

A minimum size for each leaf

Python Code

```
grid = {"min_samples_leaf": [20, 50, 100, 200, 400]}
search = GridSearchCV(DecisionTreeRegressor(random_state=0), grid, cv=folds,
                      scoring="neg_root_mean_squared_error")
search.fit(train[pair], train["vol_next"])
print(search.best_params_, round(-search.best_score_, 4))
print(search.best_estimator_.get_depth(), search.best_estimator_.get_n_leaves())
```

```
{'min_samples_leaf': 100} 0.6122
6 14
```

A cut is allowed only if both boxes keep at least `min_samples_leaf` days. At 100 the folds' error is 0.612, and the tree has 14 leaves and depth 6.

Pruning: a price for every leaf

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

Python Code

```
grid = {"ccp_alpha": [0.001, 0.002, 0.005, 0.01, 0.02, 0.05]}
search = GridSearchCV(DecisionTreeRegressor(random_state=0), grid, cv=folds,
                      scoring="neg_root_mean_squared_error")
search.fit(train[pair], train["vol_next"])
print(search.best_params_, search.best_estimator_.get_n_leaves())
```

```
{'ccp_alpha': 0.02} 4
```

$R(T)$ is the mean squared error left in the leaves of tree T , and $|T|$ counts them; as in ridge, α prices size. The folds choose 0.02: four leaves.

6 • Bagging and random forests

PART 6

Why one deep tree is not enough, and two ways of averaging many.

Two halves, two trees

Python Code

```
half_a = train.sample(frac=0.5, random_state=0)
half_b = train.drop(half_a.index)
tree_a = DecisionTreeRegressor(random_state=0).fit(half_a[columns], half_a["vol_next"])
tree_b = DecisionTreeRegressor(random_state=0).fit(half_b[columns], half_b["vol_next"])
p_a, p_b = tree_a.predict(test[columns]), tree_b.predict(test[columns])
print(round(np.abs(p_a - p_b).mean(), 3))
print(round(rmse(test["vol_next"], p_a), 3), round(rmse(test["vol_next"], p_b), 3),
        round(rmse(test["vol_next"], (p_a + p_b) / 2), 3))
```

0.387

0.408 0.428 0.329

Two full trees, each grown on a random half of the training days, differ by 0.39 on the average test day. The average of the two scores 0.329, better than either.

Why averaging helps

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B \hat{f}^b(\mathbf{x}) \right) = \frac{\sigma^2}{B}$$

If B trees grown on independent samples each forecast with variance σ^2 , their average has variance σ^2/B . There is one training set, so the samples come from it.

A deep tree has low bias and high variance, and variance is the part an average removes.

A bootstrap sample

Python Code

```
sample = train.sample(n=len(train), replace=True, random_state=0)

print(len(sample), sample.index.nunique())
print(round(sample.index.nunique() / len(train), 3))
```

```
1894 1207
0.637
```

replace=True puts each drawn day back, and **nunique** counts the distinct days. A day is missed by all n draws with probability $(1 - 1/n)^n \approx e^{-1} = 0.37$, so about 63 percent are in.

Bagging by hand

Python Code

```
total = np.zeros(len(test))
for b in range(20):
    sample = train.sample(n=len(train), replace=True, random_state=b)
    grown = DecisionTreeRegressor(random_state=0).fit(sample[columns], sample["vol_next"])
    total = total + grown.predict(test[columns])

print(round(rmse(test["vol_next"], total / 20), 4))
```

0.32

Each tree grows fully on its own bootstrap sample, and the average is $\frac{1}{B} \sum_b \hat{f}^b(\mathbf{x})$ with $B = 20$. This is bagging, from bootstrap aggregating: 0.320, where one full tree scored 0.410.

The first question of each bagged tree

Python Code

```
from sklearn.ensemble import BaggingRegressor
bag = BaggingRegressor(DecisionTreeRegressor(), n_estimators=20, random_state=0)
bag.fit(train[columns], train["vol_next"])
first = []
for t in bag.estimators_:
    first.append(columns[t.tree_.feature[0]])
print(pd.Series(first).value_counts())
```

```
vol_5d      11
DIS_vol      5
XOM_vol      2
vol_10d      1
AAPL_vol     1
Name: count, dtype: int64
```

Averaging trees that agree

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B \hat{f}^b(\mathbf{x}) \right) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2$$

ρ is the correlation between two trees' forecasts. More trees shrink only the second term; the first stays until the trees are made less alike.

A random forest

Python Code

```
from sklearn.ensemble import RandomForestRegressor

forest = RandomForestRegressor(n_estimators=100, max_features="sqrt",
                              oob_score=True, random_state=0)
forest.fit(train[columns], train["vol_next"])

print(round(rmse(test["vol_next"], forest.predict(test[columns])), 4))
```

0.2835

`max_features="sqrt"` lets each question choose among 4 of the 19 columns, drawn at random. Without it the regressor uses every column, which is bagging.

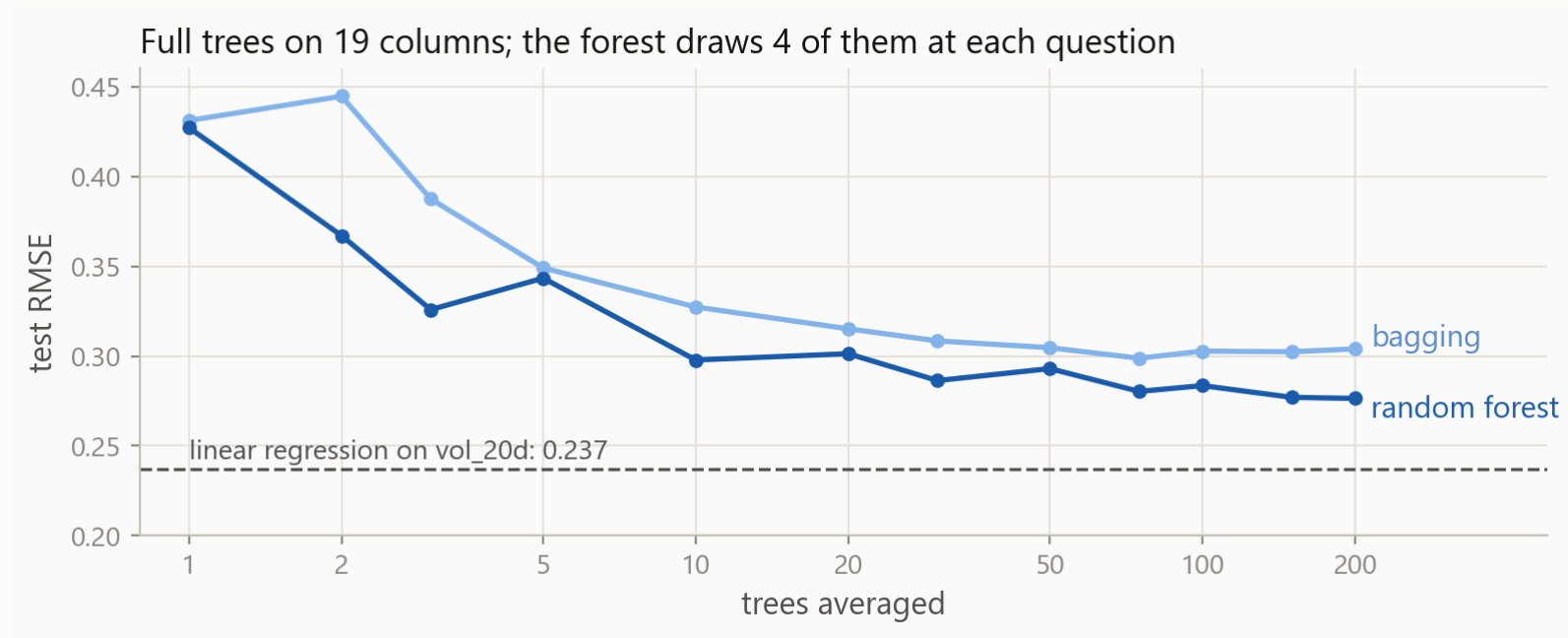
The first questions in a forest

Python Code

```
first = []
for t in forest.estimators_:
    first.append(columns[t.tree_.feature[0]])
counts = pd.Series(first).value_counts()
print(len(counts))
print(counts.head(4))
```

```
12
vol_10d      22
vol_5d       18
XOM_vol      11
MSFT_vol     10
Name: count, dtype: int64
```

The number of trees



The forest's average errs less than bagging's at every size: 0.284 against 0.303 at 100 trees. Both flatten after about 50, above linear regression.

Out-of-bag rows on a time series

Python Code

```
from sklearn.model_selection import cross_val_score
print(round(rmse(train["vol_next"], forest.oob_prediction_), 4))
scores = cross_val_score(forest, train[colums], train["vol_next"], cv=folds,
                          scoring="neg_root_mean_squared_error")
print(round(-scores.mean(), 4))
```

0.2519

0.7232

Each day is forecast by the trees whose sample left it out: an error of 0.252. On folds that score only later days it is 0.723, against 0.560 for ridge.

A day left out of a tree has its neighbours in the bag, and their next 20 days overlap its own by 19. On days, choose settings on **TimeSeriesSplit** folds.

Your turn 3

Build a forest like `forest`, but with leaves of at least 100 days, and print its test RMSE. Is it better than the forest of full trees?

Python Code

```
big_leaves = ...  
big_leaves.fit(train[columns], train["vol_next"])  
print(round(rmse(test["vol_next"], big_leaves.predict(test[columns])), 4))
```

`min_samples_leaf` sets the smallest leaf, as it did for one tree. Keep the other settings of `forest`.

Your turn 3: answer

```
big_leaves = RandomForestRegressor(n_estimators=100, max_features="sqrt",  
                                   min_samples_leaf=100, random_state=0)
```

Leaves of at least 100 days score 0.242, against 0.284 for full trees. Linear regression's 0.237 is still lower.

7 • Tuning and its cost

PART 7

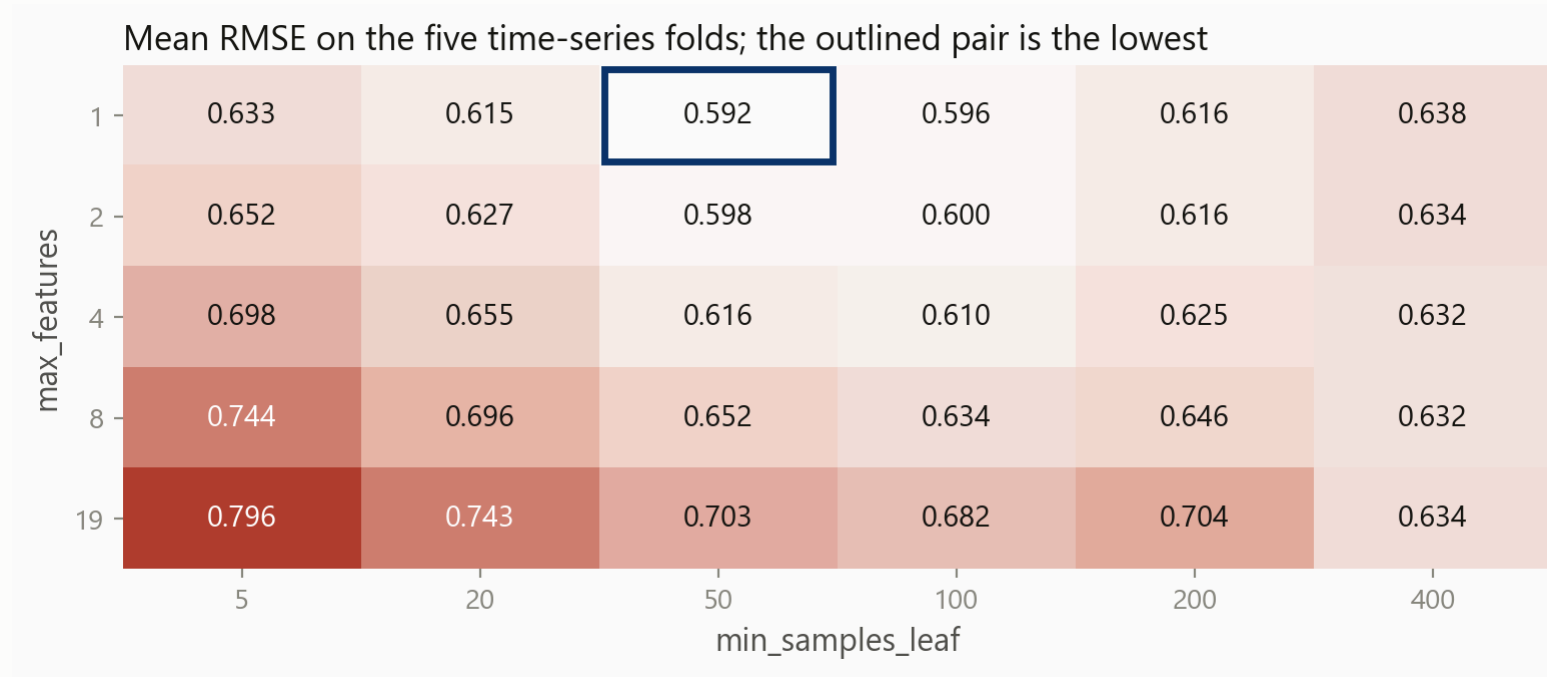
Why settings matter more for trees, what a search costs, and model size.

Settings to choose, model by model

model	settings that change the fit	scikit-learn's default	test RMSE, default and tuned
linear regression	none		0.237
ridge	<code>alpha</code>	1.0	0.248, then 0.218
a tree, on two columns	<code>max_depth</code> , <code>min_samples_leaf</code> , <code>ccp_alpha</code> , and more	no limit: one leaf per day	0.466, then 0.241
a random forest	the tree's settings, plus <code>n_estimators</code> and <code>max_features</code>	100 full trees on every column	0.303, then 0.233

Ridge has one setting, and tuning it moved the error by 0.030. A forest's defaults grow every tree down to single days, and tuning moved it by 0.070.

A grid over two forest settings



GridSearchCV scored 30 pairs on the five folds, 150 forests. One column per question and leaves of 50 days win: 0.233 on the test days, below linear regression's 0.237.

What a search costs

search, on five time-series folds	combinations	fitted	time on one core
ridge: 7 values of <code>alpha</code>	7	35 models	0.1 s
a tree: 6 depths	6	30 trees	0.2 s
a tree: 6 depths, 5 leaf sizes, 6 values of <code>ccp_alpha</code>	180	900 trees	5 s
a forest: 5 <code>max_features</code> , 6 leaf sizes, 100 trees each	30	150 forests, 15,000 trees	26 s
the same, times 3 tree counts and 4 depths	360	1,800 forests, 540,000 trees	14 min

Every setting multiplies the grid, and the folds multiply it again. The last search took 32 times as long and chose 100 trees of depth 3: 0.226 on the test days.

`RandomizedSearchCV(model, grid, n_iter=20, ...)` tries 20 combinations drawn at random, when the whole grid would take too long.

Timing one fit

Python Code

```
import time

start = time.perf_counter()
forest.fit(train[columns], train["vol_next"])
seconds = time.perf_counter() - start

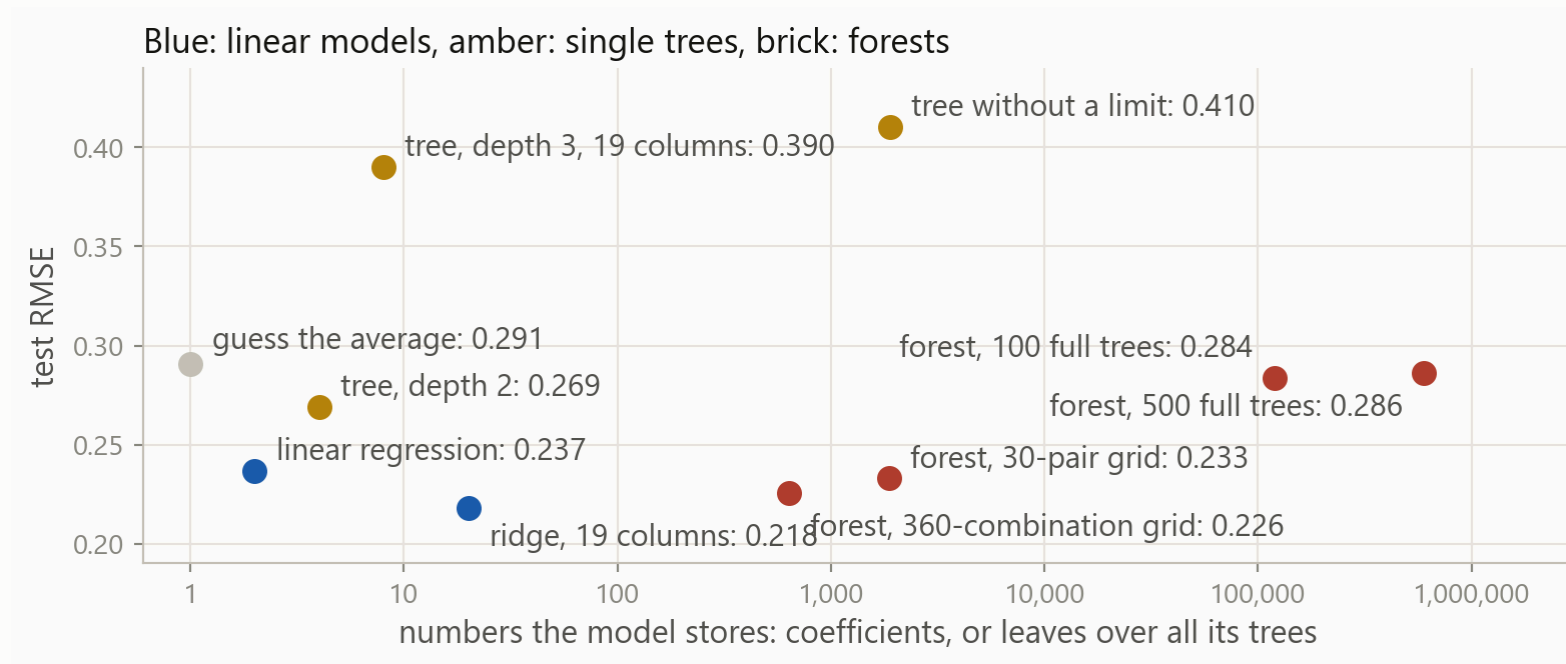
print(round(seconds, 2))
print(round(seconds * 150 / 60, 1))
```

0.69

1.7

`time.perf_counter()` reads a clock, so the difference is the seconds one forest took here. The grid of 30 pairs fits 150 forests; the last line is its minutes.

Model size against test error



The largest model, 500 full trees with 598,735 leaves, took about 1,800 times longer to fit than ridge and forecasts worse than its 20 coefficients.

8 • A tree for a label

PART 8

The same questions on the credit table, with a share in each leaf.

The credit table

Python Code

```
from sklearn.model_selection import train_test_split

credit = pd.read_csv("data/credit.csv")
c_train, c_test = train_test_split(credit, test_size=0.3, random_state=0,
                                   stratify=credit["default"])
c_columns = ["limit", "age", "late_now", "months_late", "bill", "paid", "utilisation"]
print(len(c_train), len(c_test), round(c_train["default"].mean(), 3))
```

```
21000 9000 0.221
```

One row per card holder, and **default** is 1 for the 22 percent who defaulted the next month. On this split logistic regression scored an AUC of 0.748.

A classification tree

Python Code

```
from sklearn.tree import DecisionTreeClassifier
c_tree = DecisionTreeClassifier(max_depth=2, random_state=0)
c_tree.fit(c_train[c_columns], c_train["default"])
plt.figure(figsize=(10, 3))
plot_tree(c_tree, feature_names=c_columns, filled=True, proportion=True,
          impurity=False, fontsize=10)
plt.show()
```

Gini, for labels

$$G = \sum_k \hat{p}_k (1 - \hat{p}_k)$$

For labels of 0 and 1, a box of n rows with k defaults has an RSS of $k(n - k)/n$.
With two classes $G = 2p(1 - p)$, twice that per row, so the same cuts win.

`DecisionTreeClassifier` measures a box with Gini by default, and `criterion="entropy"` switches to $D = - \sum_k \hat{p}_k \log \hat{p}_k$.

A forest on the credit table

Python Code

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

c_forest = RandomForestClassifier(n_estimators=100, min_samples_leaf=100,
                                oob_score=True, random_state=0)
c_forest.fit(c_train[c_columns], c_train["default"])
oob = c_forest.oob_decision_function[:, 1]
p_forest = c_forest.predict_proba(c_test[c_columns])[:, 1]
print(round(roc_auc_score(c_train["default"], oob), 4))
print(round(roc_auc_score(c_test["default"], p_forest), 4))
```

0.7806

0.7766

The rows are separate borrowers, so the out-of-bag score can be trusted: 0.781. On the test rows the forest scores 0.777, against 0.748 for logistic regression.

When to reach for a tree

The credit table

21,000 separate borrowers, a share of defaults that levels off, and a reversal that depends on another column. The forest beat logistic regression, 0.777 against 0.748.

The volatility table

1,894 days whose windows overlap, a relationship close to a straight line, and calm test years. Ridge beat every tree, 0.218 against 0.226 for the best-tuned forest.

Fit a linear model first, and make the tree beat it on folds that suit the rows.

When to change what

Always

the size of each tree: `max_depth` or `min_samples_leaf`, on `TimeSeriesSplit` folds for days, or out-of-bag for a forest of separate rows. `random_state`, which fixes the samples and the order in which columns are tried.

Rarely

`criterion`, the error a cut is judged by.
`max_leaf_nodes`, a cap on the number of leaves.

Sometimes

`ccp_alpha`, to prune a grown tree. `n_estimators`: more trees only cost time. `max_features`, the columns each question chooses among: `"sqrt"` makes a regressor a forest. `class_weight`, for labels. `n_jobs=-1`, to fit a forest's trees on every core.

Never needed

A scaler. A cut compares one column with one number, so `StandardScaler` in front of a tree gives the same forecasts.

What stays with you

Linear regression has one slope

When volatility was high, linear regression on `vol_20d` was 0.18 too low after a fall and 0.14 too high after a rise.

Grown greedily, one cut at a time

Every column and every cut, the smallest RSS, then again inside each box; a stump is one step. The best number for a box is its mean.

Depth is the setting

Without a limit: one leaf per training day, an error of 0 on them and 0.466 on the test days. Choose the size of a tree on `TimeSeriesSplit` folds.

Tuning, cost and size

Tuning moved the forest by 0.070 and ridge by 0.030. Each setting multiplies a grid, and 500 full trees forecast worse than ridge's 20 coefficients.

A tree is questions, boxes and a formula

$\hat{f}(\mathbf{x}) = \sum_m c_m \mathbf{1}\{\mathbf{x} \in R_m\}$: each leaf forecasts the mean of its training days.

`plot_tree` shows it

`plot_tree(model, feature_names=...)`, with `filled`, `impurity=False`, `precision` and `label="root"`. Each box shows its question, error, days and forecast.

Bagging and forests, for variance

Averaging 20 bootstrap trees took the error from 0.410 to 0.320. Drawing columns at random makes the trees differ, and a tuned forest reached 0.226.

A tree for a label

The same cuts, judged by Gini. On the credit table the forest beat logistic regression; on the volatility table linear regression and ridge beat every tree.

Where this goes next

before

linear regression, ridge and lasso for numbers; logistic regression and k-nearest neighbours for labels

formulas and neighbours

now

trees, and forests that average them, for numbers and for labels

questions, averaged

next

boosting: small trees grown one after another, each fitted to the errors so far, in scikit-learn, XGBoost and LightGBM; reading what a model has learned; the course as a whole

the wrap-up

Bagging and forests grow deep trees side by side and average away their variance. Boosting grows small trees in sequence, each correcting what the ones before it still get wrong, which removes bias.

Before the next session **CASE**

- **Exercises:** cuts scored by hand, stumps and trees drawn with `plot_tree`, depth and pruning on the time-series folds, bootstrap samples, bagging and forests, grids counted and timed, and a tree for a label.
- **Case Part 11:** the risk report's volatility forecast gets a tree and a forest, held to its one-column model and to ridge on Apple and across the desk; a last question tries the forest on Part 9's jump label.

Stuck? Ask a friend, ask an AI for a hint rather than an answer, or email jobo@econ.au.dk.